AI Driven Full Stack Development (AI308B)

Moodle ESE AIML Submission Format

Name	Kartik Upadhyay
Branch	CSE (AI&ML)
Roll Number	2500291539005
Section	B
Shift	Evening
Case Study Name	AI-Based Smart Complaint Management System

GitHub Repository Link

Paste GitHub repository link after pushing the project.

Render Deployment Links for all Routes

Backend Deployment Link	Paste backend Render URL after deployment
Frontend Deployment Link	Paste frontend Render URL after deployment
POST /api/register	https://your-backend.onrender.com/api/register
POST /api/login	https://your-backend.onrender.com/api/login
POST /api/complaints	https://your-backend.onrender.com/api/complaints
GET /api/complaints	https://your-backend.onrender.com/api/complaints
GET /api/complaints/search?location=Ghaziabad	https://your-backend.onrender.com/api/complaints/search?location=Ghaziabad
PUT /api/complaints/:id	https://your-backend.onrender.com/api/complaints/:id
DELETE /api/complaints/:id	https://your-backend.onrender.com/api/complaints/:id
POST /api/ai/analyze	https://your-backend.onrender.com/api/ai/analyze

Project Code

Backend main file is server.js. Real .env file is not included because it contains MongoDB Atlas password and API keys. The safe .env.example file is shown.

Backend Code

Backend Code (server.js, .git, .env, model.js etc.) which is used in the project development.

server.js

`backend\server.js`

```
const cors = require("cors");
const dotenv = require("dotenv");
const express = require("express");
const connectDB = require("./config/db");
const { errorHandler, notFound } = require("./middleware/errorMiddleware");
const aiRoutes = require("./routes/aiRoutes");
const authRoutes = require("./routes/authRoutes");
const complaintRoutes = require("./routes/complaintRoutes");

dotenv.config();

connectDB();

const app = express();
const PORT = process.env.PORT || 5000;

app.use(cors());
app.use(express.json());

app.get("/", (req, res) => {
  res.json({ message: "AI-Based Smart Complaint Management API is running" });
});

app.use("/api", authRoutes);
app.use("/api/complaints", complaintRoutes);
app.use("/api/ai", aiRoutes);

app.use(notFound);
app.use(errorHandler);

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

config/db.js

`backend\config\db.js`

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    const connection = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB connected: ${connection.connection.host}`);
  } catch (error) {
    console.error(`MongoDB connection failed: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

.env.example

`backend\.env.example`

```
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@cluster.mongodb.net/smart_complaints
JWT_SECRET=smart_complaint_jwt_secret_2026
```

```
OPENROUTER_API_KEY=your_openrouter_api_key_here
OPENROUTER_MODEL=openai/gpt-4o-mini
```

models/User.js

backend\models\User.js

```
const bcrypt = require("bcrypt");
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, "Name is required"],
      trim: true
    },
    email: {
      type: String,
      required: [true, "Email is required"],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, "Please enter a valid email"]
    },
    password: {
      type: String,
      required: [true, "Password is required"],
      minlength: [6, "Password must be at least 6 characters"]
    },
    role: {
      type: String,
      enum: ["User", "Admin"],
      default: "Admin"
    }
  },
  { timestamps: true }
);

userSchema.pre("save", async function hashPassword(next) {
  if (!this.isModified("password")) {
    return next();
  }

  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

userSchema.methods.comparePassword = function comparePassword(candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};

module.exports = mongoose.model("User", userSchema);
```

models/Complaint.js

backend\models\Complaint.js

```
const mongoose = require("mongoose");

const complaintSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, "Name is required"],
      trim: true
    },
    email: {
      type: String,
      required: [true, "Email is required"],
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, "Please enter a valid email"]
    },
    title: {
      type: String,
      required: [true, "Complaint title is required"],
      trim: true
    },
    description: {
      type: String,
```

```
      required: [true, "Complaint description is required"],
      trim: true
    },
    category: {
      type: String,
      required: [true, "Complaint category is required"],
      trim: true
    },
    location: {
      type: String,
      required: [true, "Location is required"],
      trim: true
    },
    status: {
      type: String,
      enum: ["Pending", "In Progress", "Resolved", "Rejected"],
      default: "Pending"
    },
    aiAnalysis: {
      urgency: { type: String, default: "Medium" },
      department: { type: String, default: "General Administration" },
      summary: { type: String, default: "" },
      response: { type: String, default: "" }
    },
    createdBy: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User"
    }
  },
  { timestamps: true }
);

module.exports = mongoose.model("Complaint", complaintSchema);
```

middleware/authMiddleware.js

backend\middleware\authMiddleware.js

```
const jwt = require("jsonwebtoken");
const User = require("../models/User");

const protect = async (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Access denied. No token provided." });
  }

  try {
    const token = authHeader.split(" ")[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id).select("-password");

    if (!req.user) {
      return res.status(401).json({ message: "Access denied. User not found." });
    }

    next();
  } catch (error) {
    return res.status(401).json({ message: "Invalid or expired token." });
  }
};

module.exports = protect;
```

middleware/errorMiddleware.js

backend\middleware\errorMiddleware.js

```
const notFound = (req, res, next) => {
  const error = new Error(`Route not found - ${req.originalUrl}`);
  res.status(404);
  next(error);
};

const errorHandler = (error, req, res, next) => {
  const statusCode = res.statusCode === 200 ? 500 : res.statusCode;

  if (error.name === "ValidationError") {
    return res.status(400).json({
      message: Object.values(error.errors)
        .map((item) => item.message)
```

```
        .join(", ")
    });
  }

  if (error.code === 11000) {
    return res.status(409).json({ message: "Duplicate record found." });
  }

  res.status(statusCode).json({
    message: error.message || "Server error"
  });
};

module.exports = { errorHandler, notFound };
```

controllers/authController.js

backend\controllers\authController.js

```
const jwt = require("jsonwebtoken");
const User = require("../models/User");

const createToken = (userId) => {
  return jwt.sign({ id: userId }, process.env.JWT_SECRET, {
    expiresIn: "7d"
  });
};

const registerUser = async (req, res, next) => {
  try {
    const { name, email, password, role } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({ message: "Please fill all required fields." });
    }

    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(409).json({ message: "User already exists." });
    }

    const user = await User.create({ name, email, password, role });

    res.status(201).json({
      _id: user._id,
      name: user.name,
      email: user.email,
      role: user.role,
      token: createToken(user._id)
    });
  } catch (error) {
    next(error);
  }
};

const loginUser = async (req, res, next) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ message: "Please enter email and password." });
    }

    const user = await User.findOne({ email });
    if (!user || !(await user.comparePassword(password))) {
      return res.status(401).json({ message: "Invalid email or password." });
    }

    res.json({
      _id: user._id,
      name: user.name,
      email: user.email,
      role: user.role,
      token: createToken(user._id)
    });
  } catch (error) {
    next(error);
  }
};

module.exports = { loginUser, registerUser };
```

controllers/complaintController.js

backend\controllers\complaintController.js

```
const Complaint = require("../models/Complaint");
const { analyzeComplaintText } = require("./aiController");

const createComplaint = async (req, res, next) => {
  try {
    const complaint = await Complaint.create({
      ...req.body,
      createdBy: req.user?._id
    });

    res.status(201).json({
      message: "Complaint stored successfully.",
      complaint
    });
  } catch (error) {
    next(error);
  }
};

const getComplaints = async (req, res, next) => {
  try {
    const { category, status } = req.query;
    const query = {};

    if (category) {
      query.category = new RegExp(category, "i");
    }

    if (status) {
      query.status = status;
    }

    const complaints = await Complaint.find(query).sort({ createdAt: -1 });
    res.json(complaints);
  } catch (error) {
    next(error);
  }
};

const searchComplaints = async (req, res, next) => {
  try {
    const { location, category } = req.query;
    const query = {};

    if (location) {
      query.location = new RegExp(location, "i");
    }

    if (category) {
      query.category = new RegExp(category, "i");
    }

    const complaints = await Complaint.find(query).sort({ createdAt: -1 });
    res.json(complaints);
  } catch (error) {
    next(error);
  }
};

const updateComplaintStatus = async (req, res, next) => {
  try {
    const complaint = await Complaint.findByIdAndUpdate(
      req.params.id,
      { status: req.body.status },
      { new: true, runValidators: true }
    );

    if (!complaint) {
      return res.status(404).json({ message: "Complaint not found." });
    }

    res.json({
      message: "Complaint status updated successfully.",
      complaint
    });
  } catch (error) {
    next(error);
  }
};

const deleteComplaint = async (req, res, next) => {
  try {
```

```
    const complaint = await Complaint.findByIdAndDelete(req.params.id);

    if (!complaint) {
      return res.status(404).json({ message: "Complaint not found." });
    }

    res.json({ message: "Complaint removed successfully." });
  } catch (error) {
    next(error);
  }
};

const analyzeComplaint = async (req, res, next) => {
  try {
    const complaint = await Complaint.findById(req.params.id);

    if (!complaint) {
      return res.status(404).json({ message: "Complaint not found." });
    }

    const aiAnalysis = await analyzeComplaintText(complaint);
    complaint.aiAnalysis = aiAnalysis;
    await complaint.save();

    res.json({
      message: "AI complaint analysis completed.",
      complaint
    });
  } catch (error) {
    next(error);
  }
};

module.exports = {
  analyzeComplaint,
  createComplaint,
  deleteComplaint,
  getComplaints,
  searchComplaints,
  updateComplaintStatus
};
```

controllers/aiController.js

backend\controllers\aiController.js

```
const keywordFallback = (complaint) => {
  const text = `${complaint.title} ${complaint.description} ${complaint.category}`.toLowerCase();
  let urgency = "Medium";
  let department = "General Administration";

  if (text.includes("electric") || text.includes("fire") || text.includes("danger")) {
    urgency = "High";
    department = "Electricity Department";
  } else if (text.includes("water") || text.includes("leak") || text.includes("pipeline")) {
    urgency = "High";
    department = "Water Supply Department";
  } else if (text.includes("garbage") || text.includes("waste") || text.includes("clean")) {
    urgency = "Medium";
    department = "Sanitation Department";
  } else if (text.includes("road") || text.includes("traffic") || text.includes("street")) {
    department = "Public Works Department";
  }

  return {
    urgency,
    department,
    summary: `${complaint.title}: ${complaint.description.slice(0, 120)}${
      complaint.description.length > 120 ? "..." : ""
    }`,
    response: `Your complaint has been registered and forwarded to the ${department}. Current priority is ${urgency}.`
  };
};

const parseAiJson = (content, complaint) => {
  try {
    const jsonStart = content.indexOf("{");
    const jsonEnd = content.lastIndexOf("}");
    const jsonText = content.slice(jsonStart, jsonEnd + 1);
    const parsed = JSON.parse(jsonText);

    return {
      urgency: parsed.urgency || "Medium",
      department: parsed.department || "General Administration",
```

```
      summary: parsed.summary || complaint.description,
      response: parsed.response || "Your complaint has been received."
    };
  } catch (error) {
    return keywordFallback(complaint);
  }
};

const analyzeComplaintText = async (complaint) => {
  if (!process.env.OPENROUTER_API_KEY) {
    return keywordFallback(complaint);
  }

  try {
    const response = await fetch("https://openrouter.ai/api/v1/chat/completions", {
      method: "POST",
      headers: {
        Authorization: `Bearer ${process.env.OPENROUTER_API_KEY}`,
        "Content-Type": "application/json",
        "HTTP-Referer": "http://localhost:5173",
        "X-Title": "Smart Complaint Management System"
      },
      body: JSON.stringify({
        model: process.env.OPENROUTER_MODEL || "openai/gpt-4o-mini",
        messages: [
          {
            role: "system",
            content:
              "Analyze civic complaints. Return only JSON with urgency, department, summary, and response."
          },
          {
            role: "user",
            content: JSON.stringify({
              title: complaint.title,
              description: complaint.description,
              category: complaint.category,
              location: complaint.location
            })
          }
        ],
        temperature: 0.2
      })
    });

    if (!response.ok) {
      return keywordFallback(complaint);
    }

    const data = await response.json();
    const content = data.choices?.[0]?.message?.content || "";
    return parseAiJson(content, complaint);
  } catch (error) {
    return keywordFallback(complaint);
  }
};

const analyzeComplaintFromBody = async (req, res, next) => {
  try {
    const analysis = await analyzeComplaintText(req.body);
    res.json({ message: "AI analysis completed.", analysis });
  } catch (error) {
    next(error);
  }
};

module.exports = { analyzeComplaintFromBody, analyzeComplaintText };
```

routes/authRoutes.js

backend\routes\authRoutes.js

```
const express = require("express");
const { loginUser, registerUser } = require("../controllers/authController");

const router = express.Router();

router.post("/signup", registerUser);
router.post("/register", registerUser);
router.post("/login", loginUser);

module.exports = router;
```

routes/complaintRoutes.js

backend\routes\complaintRoutes.js

```
const express = require("express");
const {
  analyzeComplaint,
  createComplaint,
  deleteComplaint,
  getComplaints,
  searchComplaints,
  updateComplaintStatus
} = require("../controllers/complaintController");
const protect = require("../middleware/authMiddleware");

const router = express.Router();

router.use(protect);

router.post("/", createComplaint);
router.get("/", getComplaints);
router.get("/search", searchComplaints);
router.put("/:id", updateComplaintStatus);
router.delete("/:id", deleteComplaint);
router.post("/:id/analyze", analyzeComplaint);

module.exports = router;
```

routes/aiRoutes.js

backend\routes\aiRoutes.js

```
const express = require("express");
const { analyzeComplaintFromBody } = require("../controllers/aiController");
const protect = require("../middleware/authMiddleware");

const router = express.Router();

router.post("/analyze", protect, analyzeComplaintFromBody);

module.exports = router;
```

Frontend Code

Frontend Code (App.jsx, main.jsx, Dashboard.jsx, Login.jsx, Register.jsx etc.) which are used in the project development.

App.jsx

frontend\src\App.jsx

```
import { Navigate, Route, Routes } from "react-router-dom";
import ProtectedRoute from "./components/ProtectedRoute.jsx";
import Dashboard from "./pages/Dashboard.jsx";
import Login from "./pages/Login.jsx";
import Register from "./pages/Register.jsx";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Navigate to="/login" replace />} />
      <Route path="/register" element={<Register />} />
      <Route path="/login" element={<Login />} />
      <Route
        path="/dashboard"
        element={
          <ProtectedRoute>
            <Dashboard />
          </ProtectedRoute>
        }
      />
    </Routes>
  );
}

export default App;
```

main.jsx

frontend\src\main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App.jsx";
import { AuthProvider } from "./context/AuthContext.jsx";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

api.js

frontend\src\api.js

```
const API_URL = import.meta.env.VITE_API_URL || "http://localhost:5000/api";

export const apiRequest = async (path, options = {}) => {
  const auth = JSON.parse(localStorage.getItem("smartComplaintAuth") || "null");
  const headers = {
    "Content-Type": "application/json",
    ...options.headers
  };

  if (auth?.token) {
    headers.Authorization = `Bearer ${auth.token}`;
  }

  const response = await fetch(`${API_URL}${path}`, {
    ...options,
    headers
```

```
  });

  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message || "Something went wrong");
  }

  return data;
};
```

context/AuthContext.jsx

frontend\src\context\AuthContext.jsx

```
import { createContext, useContext, useMemo, useState } from "react";

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [auth, setAuth] = useState(() => {
    return JSON.parse(localStorage.getItem("smartComplaintAuth") || "null");
  });

  const login = (userData) => {
    localStorage.setItem("smartComplaintAuth", JSON.stringify(userData));
    setAuth(userData);
  };

  const logout = () => {
    localStorage.removeItem("smartComplaintAuth");
    setAuth(null);
  };

  const value = useMemo(
    () => ({
      auth,
      isAuthenticated: Boolean(auth?.token),
      login,
      logout
    }),
    [auth]
  );

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
};

export const useAuth = () => useContext(AuthContext);
```

components/ProtectedRoute.jsx

frontend\src\components\ProtectedRoute.jsx

```
import { Navigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext.jsx";

const ProtectedRoute = ({ children }) => {
  const { isAuthenticated } = useAuth();

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  return children;
};

export default ProtectedRoute;
```

Register.jsx

frontend\src\pages\Register.jsx

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { apiRequest } from "../api.js";
import { useAuth } from "../context/AuthContext.jsx";

const Register = () => {
  const navigate = useNavigate();
```

```
  const { login } = useAuth();
  const [form, setForm] = useState({
    name: "",
    email: "",
    password: "",
    role: "Admin"
  });
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const handleChange = (event) => {
    setForm({ ...form, [event.target.name]: event.target.value });
  };

  const handleSubmit = async (event) => {
    event.preventDefault();
    setError("");
    setLoading(true);

    try {
      const data = await apiRequest("/register", {
        method: "POST",
        body: JSON.stringify(form)
      });
      login(data);
      navigate("/dashboard");
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  return (
    <main className="auth-page">
      <section className="auth-panel">
        <p className="eyebrow">Smart Complaint Management</p>
        <h1>Create Account</h1>
        <p>Register an HR/Admin user to manage complaints and AI analysis.</p>

        {error && <div className="error-message">{error}</div>}

        <form className="form" onSubmit={handleSubmit}>
          <label>
            Name
            <input
              name="name"
              value={form.name}
              onChange={handleChange}
              placeholder="Enter full name"
              required
            />
          </label>

          <label>
            Email
            <input
              name="email"
              type="email"
              value={form.email}
              onChange={handleChange}
              placeholder="Enter email"
              required
            />
          </label>

          <label>
            Password
            <input
              name="password"
              type="password"
              value={form.password}
              onChange={handleChange}
              placeholder="Minimum 6 characters"
              required
            />
          </label>

          <label>
            Role
            <select name="role" value={form.role} onChange={handleChange}>
              <option value="Admin">Admin</option>
              <option value="User">User</option>
            </select>
          </label>

          <button type="submit" disabled={loading}>
```

```
            {loading ? "Creating..." : "Create Account"}
          </button>
        </form>

        <p className="switch-link">
          Already registered? <Link to="/login">Login</Link>
        </p>
      </section>
    </main>
  );
};

export default Register;
```

Login.jsx

frontend\src\pages\Login.jsx

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { apiRequest } from "../api.js";
import { useAuth } from "../context/AuthContext.jsx";

const Login = () => {
  const navigate = useNavigate();
  const { login } = useAuth();
  const [form, setForm] = useState({ email: "", password: "" });
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const handleChange = (event) => {
    setForm({ ...form, [event.target.name]: event.target.value });
  };

  const handleSubmit = async (event) => {
    event.preventDefault();
    setError("");
    setLoading(true);

    try {
      const data = await apiRequest("/login", {
        method: "POST",
        body: JSON.stringify(form)
      });
      login(data);
      navigate("/dashboard");
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  return (
    <main className="auth-page">
      <section className="auth-panel">
        <p className="eyebrow">AI-Based Complaint System</p>
        <h1>Login</h1>
        <p>Access protected complaint tracking and AI recommendation modules.</p>

        {error && <div className="error-message">{error}</div>}

        <form className="form" onSubmit={handleSubmit}>
          <label>
            Email
            <input
              name="email"
              type="text"
              value={form.email}
              onChange={handleChange}
              placeholder="Enter email"
              required
            />
          </label>

          <label>
            Password
            <input
              name="password"
              type="password"
              value={form.password}
              onChange={handleChange}
              placeholder="Enter password"
              required
```

```
            />
          </label>

          <button type="submit" disabled={loading}>
            {loading ? "Checking..." : "Login"}
          </button>
        </form>

        <p className="switch-link">
          New user? <Link to="/register">Create account</Link>
        </p>
      </section>
    </main>
  );
};

export default Login;
```

Dashboard.jsx

frontend\src\pages\Dashboard.jsx

```
import { useEffect, useMemo, useState } from "react";
import { useNavigate } from "react-router-dom";
import { apiRequest } from "../api.js";
import { useAuth } from "../context/AuthContext.jsx";

const emptyForm = {
  name: "",
  email: "",
  title: "",
  description: "",
  category: "Water Supply",
  location: "",
  status: "Pending"
};

const statusOptions = ["Pending", "In Progress", "Resolved", "Rejected"];

const Dashboard = () => {
  const navigate = useNavigate();
  const { auth, logout } = useAuth();
  const [form, setForm] = useState(emptyForm);
  const [complaints, setComplaints] = useState([]);
  const [filters, setFilters] = useState({ category: "", location: "" });
  const [selectedAnalysis, setSelectedAnalysis] = useState(null);
  const [message, setMessage] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const stats = useMemo(() => {
    return {
      total: complaints.length,
      pending: complaints.filter((item) => item.status === "Pending").length,
      resolved: complaints.filter((item) => item.status === "Resolved").length
    };
  }, [complaints]);

  const fetchComplaints = async () => {
    try {
      const query = new URLSearchParams();
      if (filters.location) query.set("location", filters.location);
      if (filters.category) query.set("category", filters.category);

      const endpoint = query.toString()
        ? `/complaints/search?${query.toString()}`
        : "/complaints";
      const data = await apiRequest(endpoint);
      setComplaints(data);
    } catch (err) {
      setError(err.message);
    }
  };

  useEffect(() => {
    fetchComplaints();
  }, []);

  const handleChange = (event) => {
    setForm({ ...form, [event.target.name]: event.target.value });
  };

  const handleSubmit = async (event) => {
    event.preventDefault();
```

```
    setError("");
    setMessage("");
    setLoading(true);

    try {
      await apiRequest("/complaints", {
        method: "POST",
        body: JSON.stringify(form)
      });
      setMessage("Complaint stored successfully.");
      setForm(emptyForm);
      await fetchComplaints();
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  const handleStatusUpdate = async (id, status) => {
    setError("");
    setMessage("");

    try {
      await apiRequest(`/complaints/${id}`, {
        method: "PUT",
        body: JSON.stringify({ status })
      });
      setMessage("Complaint status updated successfully.");
      await fetchComplaints();
    } catch (err) {
      setError(err.message);
    }
  };

  const handleDelete = async (id) => {
    setError("");
    setMessage("");

    try {
      await apiRequest(`/complaints/${id}`, { method: "DELETE" });
      setMessage("Complaint removed successfully.");
      await fetchComplaints();
    } catch (err) {
      setError(err.message);
    }
  };

  const handleAnalyze = async (complaint) => {
    setError("");
    setMessage("");

    try {
      const data = await apiRequest(`/complaints/${complaint._id}/analyze`, {
        method: "POST"
      });
      setSelectedAnalysis(data.complaint);
      await fetchComplaints();
    } catch (err) {
      setError(err.message);
    }
  };

  const handleLogout = () => {
    logout();
    navigate("/login");
  };

  return (
    <main className="dashboard-page">
      <header className="dashboard-header">
        <div>
          <p className="eyebrow">Welcome, {auth?.name}</p>
          <h1>Smart Complaint Dashboard</h1>
        </div>
        <button type="button" className="secondary-button" onClick={handleLogout}>
          Logout
        </button>
      </header>

      <section className="stats-row">
        <div>
          <span>Total Complaints</span>
          <strong>{stats.total}</strong>
        </div>
        <div>
          <span>Pending</span>
```

```
          <strong>{stats.pending}</strong>
        </div>
        <div>
          <span>Resolved</span>
          <strong>{stats.resolved}</strong>
        </div>
      </section>

      {message && <div className="success-message">{message}</div>}
      {error && <div className="error-message">{error}</div>}

      <section className="dashboard-grid">
        <div className="panel">
          <h2>Complaint Registration Form</h2>
          <form className="form" onSubmit={handleSubmit}>
            <label>
              Name
              <input name="name" value={form.name} onChange={handleChange} required />
            </label>

            <label>
              Email
              <input
                name="email"
                type="email"
                value={form.email}
                onChange={handleChange}
                required
              />
            </label>

            <label>
              Complaint Title
              <input name="title" value={form.title} onChange={handleChange} required />
            </label>

            <label>
              Complaint Category
              <select name="category" value={form.category} onChange={handleChange}>
                <option>Water Supply</option>
                <option>Electricity</option>
                <option>Sanitation</option>
                <option>Roads</option>
                <option>Other</option>
              </select>
            </label>

            <label>
              Location
              <input name="location" value={form.location} onChange={handleChange} required />
            </label>

            <label>
              Complaint Description
              <textarea
                name="description"
                value={form.description}
                onChange={handleChange}
                rows="5"
                required
              />
            </label>

            <button type="submit" disabled={loading}>
              {loading ? "Submitting..." : "Submit Complaint"}
            </button>
          </form>
        </div>

        <div className="panel">
          <h2>Search & Filter Section</h2>
          <div className="filter-row">
            <input
              placeholder="Search by location"
              value={filters.location}
              onChange={(event) => setFilters({ ...filters, location: event.target.value })}
            />
            <select
              value={filters.category}
              onChange={(event) => setFilters({ ...filters, category: event.target.value })}
            >
              <option value="">All Categories</option>
              <option>Water Supply</option>
              <option>Electricity</option>
              <option>Sanitation</option>
              <option>Roads</option>
              <option>Other</option>
```

```
            </select>
            <button type="button" onClick={fetchComplaints}>
              Apply
            </button>
          </div>

          <h2>Complaint List Page</h2>
          <div className="complaint-list">
            {complaints.length === 0 ? (
              <p className="empty-text">No complaints found.</p>
            ) : (
              complaints.map((complaint) => (
                <article className="complaint-card" key={complaint._id}>
                  <div className="card-top">
                    <div>
                      <h3>{complaint.title}</h3>
                      <p>{complaint.name} · {complaint.location}</p>
                    </div>
                    <span className={`status ${complaint.status.toLowerCase().replace(" ", "-")}`}>
                      {complaint.status}
                    </span>
                  </div>
                  <p>{complaint.description}</p>
                  <div className="meta-row">
                    <span>{complaint.category}</span>
                    <span>{complaint.email}</span>
                  </div>

                  {complaint.aiAnalysis?.response && (
                    <div className="analysis-mini">
                      <strong>AI:</strong> {complaint.aiAnalysis.department} ·{" "}
                      {complaint.aiAnalysis.urgency}
                    </div>
                  )}

                  <div className="button-row">
                    <select
                      value={complaint.status}
                      onChange={(event) =>
                        handleStatusUpdate(complaint._id, event.target.value)
                      }
                    >
                      {statusOptions.map((status) => (
                        <option key={status}>{status}</option>
                      ))}
                    </select>
                    <button type="button" onClick={() => handleAnalyze(complaint)}>
                      AI Analyze
                    </button>
                    <button
                      type="button"
                      className="danger-button"
                      onClick={() => handleDelete(complaint._id)}
                    >
                      Delete
                    </button>
                  </div>
                </article>
              ))
            )}
          </div>
        </div>
      </section>

      <section className="panel ai-panel">
        <h2>AI Analysis Result Display</h2>
        {selectedAnalysis ? (
          <div className="analysis-grid">
            <div>
              <span>Urgency</span>
              <strong>{selectedAnalysis.aiAnalysis.urgency}</strong>
            </div>
            <div>
              <span>Responsible Department</span>
              <strong>{selectedAnalysis.aiAnalysis.department}</strong>
            </div>
            <div>
              <span>Complaint Summary</span>
              <p>{selectedAnalysis.aiAnalysis.summary}</p>
            </div>
            <div>
              <span>Auto Response</span>
              <p>{selectedAnalysis.aiAnalysis.response}</p>
            </div>
          </div>
        ) : (
          <p className="empty-text">Click AI Analyze on a complaint to generate results.</p>
```

```
        ))}
      </section>
    </main>
  );
};

export default Dashboard;
```

index.css

frontend\src\index.css

```
* {
  box-sizing: border-box;
}

body {
  background: #eef2f5;
  color: #17212b;
  font-family: Arial, Helvetica, sans-serif;
  margin: 0;
}

button,
input,
select,
textarea {
  font: inherit;
}

button {
  background: #176b87;
  border: 0;
  border-radius: 6px;
  color: #fff;
  cursor: pointer;
  font-weight: 700;
  padding: 11px 14px;
}

button:disabled {
  cursor: not-allowed;
  opacity: 0.65;
}

.secondary-button {
  background: #52616f;
}

.danger-button {
  background: #b42318;
}

.auth-page {
  align-items: center;
  display: flex;
  min-height: 100vh;
  justify-content: center;
  padding: 24px;
}

.auth-panel,
.panel {
  background: #fff;
  border: 1px solid #d7e0e7;
  border-radius: 8px;
  box-shadow: 0 10px 28px rgba(23, 33, 43, 0.08);
}

.auth-panel {
  max-width: 430px;
  padding: 32px;
  width: 100%;
}

.eyebrow {
  color: #176b87;
  font-size: 13px;
  font-weight: 700;
  letter-spacing: 0;
  margin: 0 0 6px;
  text-transform: uppercase;
}
```

```
h1,
h2,
h3,
p {
  margin-top: 0;
}

h1 {
  font-size: 30px;
  margin-bottom: 8px;
}

h2 {
  font-size: 20px;
  margin-bottom: 18px;
}

.form {
  display: grid;
  gap: 14px;
}

label {
  color: #3c4650;
  display: grid;
  font-size: 14px;
  font-weight: 700;
  gap: 7px;
}

input,
select,
textarea {
  border: 1px solid #c7d3dc;
  border-radius: 6px;
  color: #17212b;
  padding: 11px 12px;
  width: 100%;
}

textarea {
  resize: vertical;
}

.switch-link {
  margin: 18px 0 0;
}

.switch-link a {
  color: #176b87;
  font-weight: 700;
}

.dashboard-page {
  margin: 0 auto;
  max-width: 1180px;
  padding: 28px;
}

.dashboard-header {
  align-items: center;
  display: flex;
  gap: 20px;
  justify-content: space-between;
  margin-bottom: 20px;
}

.stats-row {
  display: grid;
  gap: 14px;
  grid-template-columns: repeat(3, 1fr);
  margin-bottom: 18px;
}

.stats-row div {
  background: #fff;
  border: 1px solid #d7e0e7;
  border-radius: 8px;
  padding: 18px;
}

.stats-row span,
.analysis-grid span {
  color: #52616f;
  display: block;
  font-size: 13px;
  font-weight: 700;
```

```
  margin-bottom: 8px;
}

.stats-row strong {
  font-size: 30px;
}

.dashboard-grid {
  display: grid;
  gap: 18px;
  grid-template-columns: 0.92fr 1.08fr;
}

.panel {
  padding: 22px;
}

.filter-row {
  display: grid;
  gap: 10px;
  grid-template-columns: 1fr 170px 90px;
  margin-bottom: 24px;
}

.complaint-list {
  display: grid;
  gap: 14px;
}

.complaint-card {
  border: 1px solid #d7e0e7;
  border-radius: 8px;
  padding: 16px;
}

.card-top,
.button-row,
.meta-row {
  align-items: center;
  display: flex;
  gap: 12px;
  justify-content: space-between;
}

.card-top p,
.meta-row {
  color: #607080;
  font-size: 13px;
}

.status {
  border-radius: 999px;
  font-size: 12px;
  font-weight: 800;
  padding: 6px 10px;
  white-space: nowrap;
}

.status.pending {
  background: #fef3c7;
  color: #92400e;
}

.status.in-progress {
  background: #dbeafe;
  color: #1d4ed8;
}

.status.resolved {
  background: #dcfce7;
  color: #166534;
}

.status.rejected {
  background: #fee2e2;
  color: #991b1b;
}

.analysis-mini {
  background: #f0f9ff;
  border-left: 4px solid #176b87;
  margin: 12px 0;
  padding: 10px;
}

.ai-panel {
  margin-top: 18px;
```

```
}

.analysis-grid {
  display: grid;
  gap: 14px;
  grid-template-columns: repeat(2, 1fr);
}

.analysis-grid div {
  background: #f7fafc;
  border: 1px solid #d7e0e7;
  border-radius: 8px;
  padding: 14px;
}

.success-message,
.error-message {
  border-radius: 6px;
  font-weight: 700;
  margin-bottom: 14px;
  padding: 12px 14px;
}

.success-message {
  background: #dcfce7;
  color: #166534;
}

.error-message {
  background: #fee2e2;
  color: #991b1b;
}

.empty-text {
  color: #607080;
}

@media (max-width: 850px) {
  .dashboard-grid,
  .stats-row,
  .analysis-grid,
  .filter-row {
    grid-template-columns: 1fr;
  }

  .dashboard-header,
  .card-top,
  .button-row,
  .meta-row {
    align-items: stretch;
    flex-direction: column;
  }
}
```

Postman / Thunder Client HTTP Request Screenshots

Endpoint	Purpose
POST /api/register	Signup with bcrypt password hashing
POST /api/login	Login and generate JWT token
POST /api/complaints	Add complaint
GET /api/complaints	View all complaints
GET /api/complaints/search?location=Ghaziabad	Search/filter complaints by location/category
PUT /api/complaints/:id	Update complaint status
DELETE /api/complaints/:id	Delete complaint
POST /api/ai/analyze	Analyze complaint text with AI API
POST /api/complaints/:id/analyze	Analyze saved complaint and store AI result

Paste Postman/Thunder Client Register Request Screenshot Here

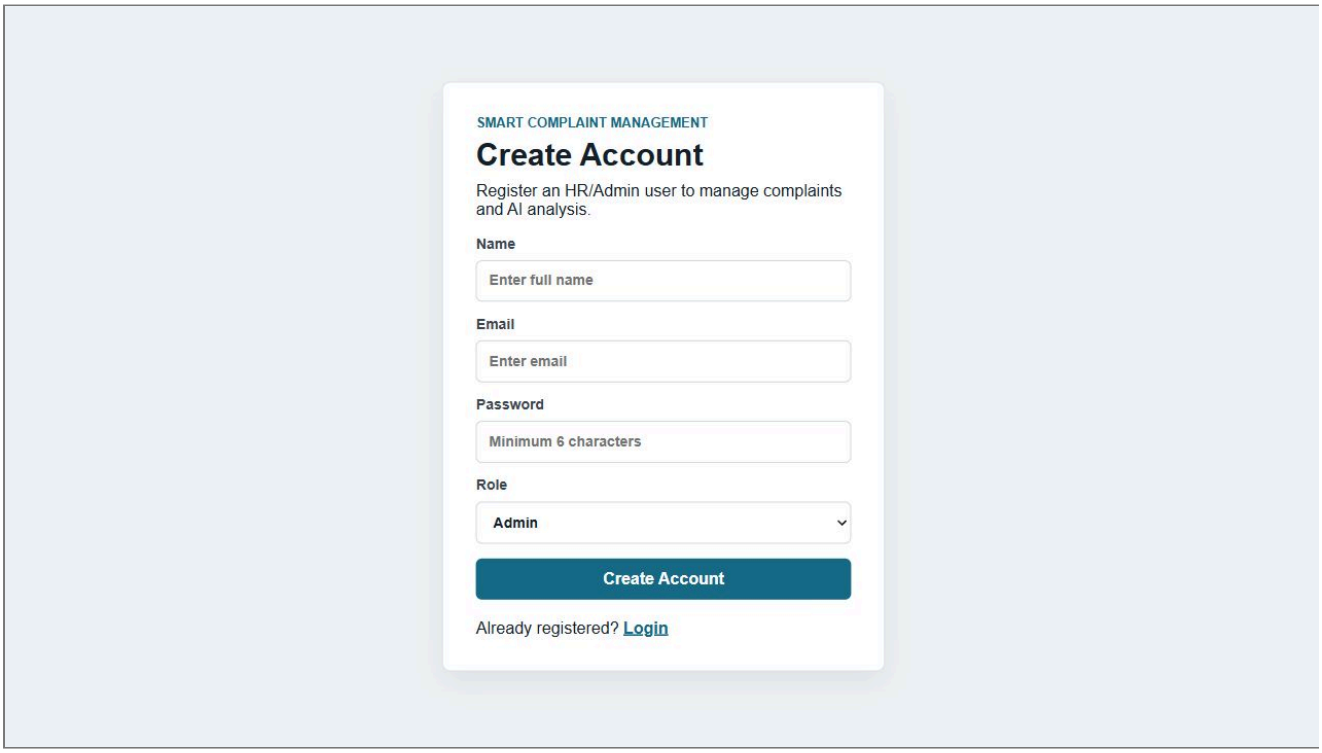
Paste Postman/Thunder Client Login Request Screenshot Here

Paste Postman/Thunder Client Complaint Request Screenshot Here

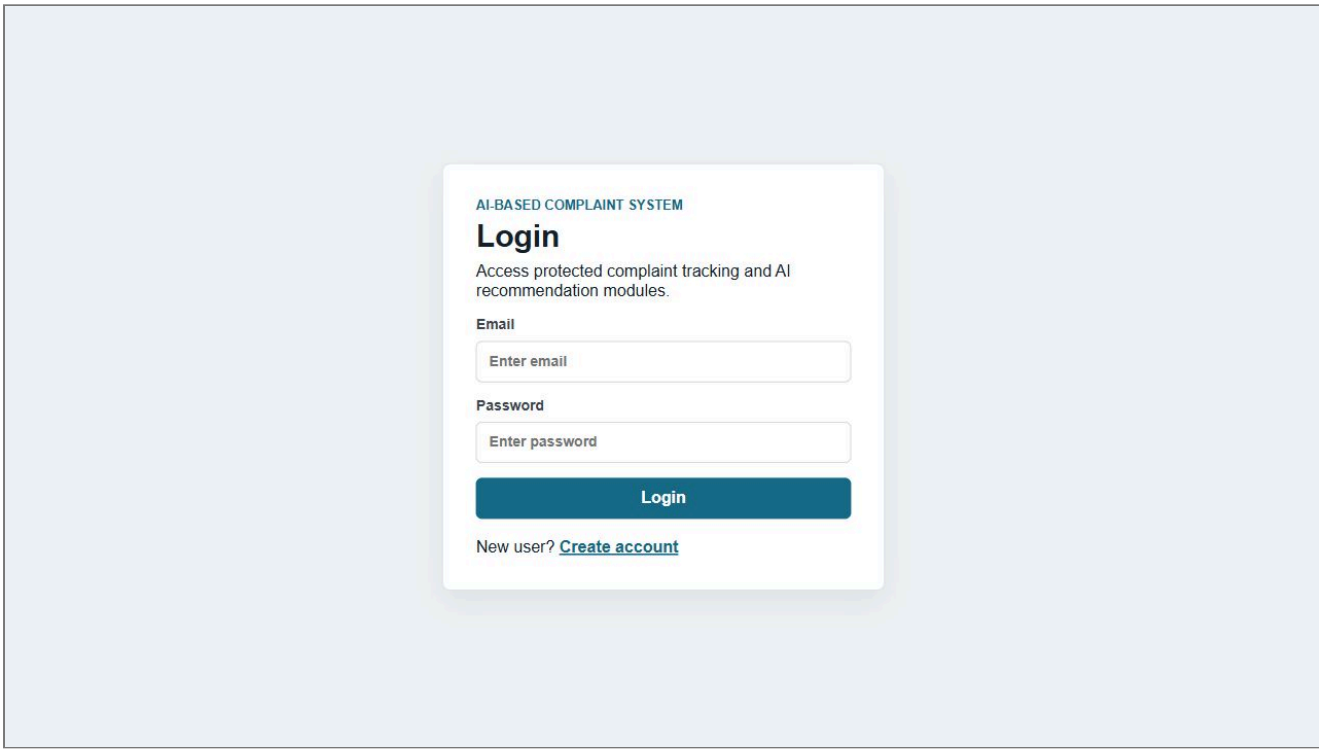
Paste Postman/Thunder Client AI Analyze Request Screenshot Here

Screenshots of Login, Register, Dashboard and All Functional Modules

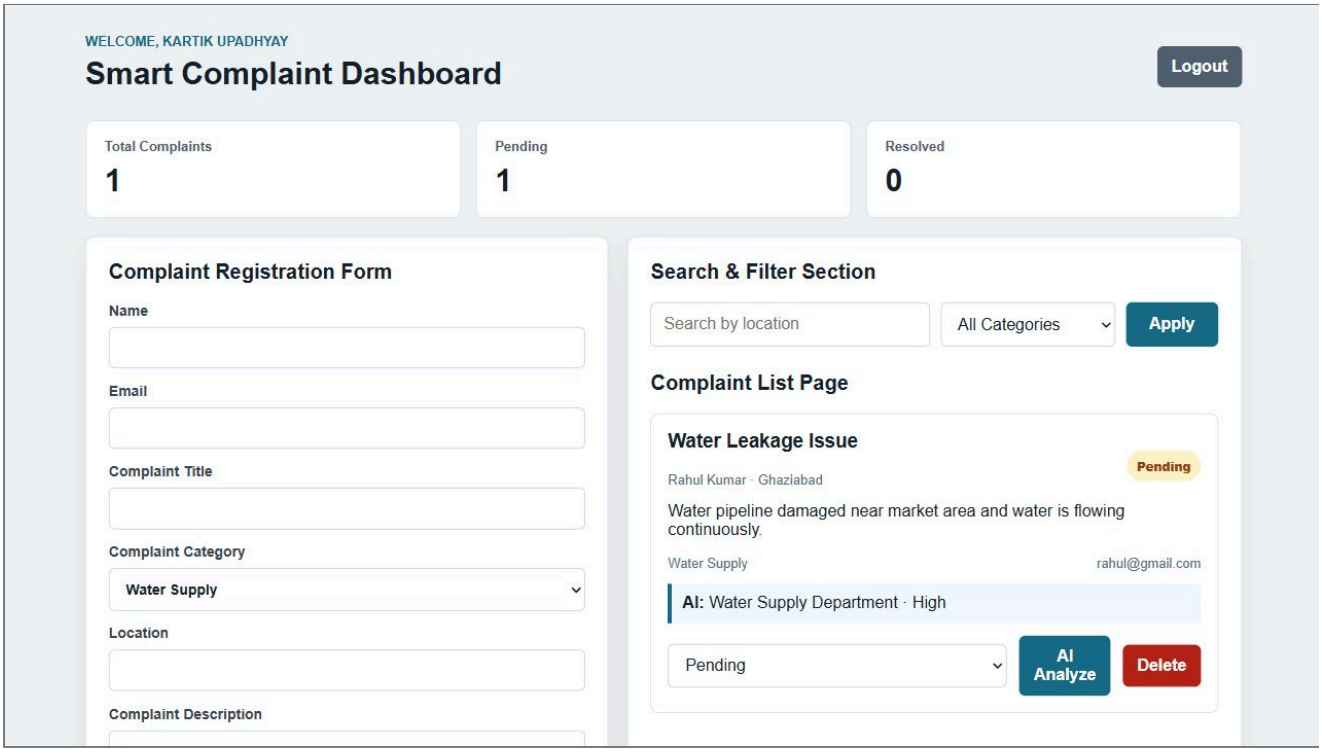
Register Page



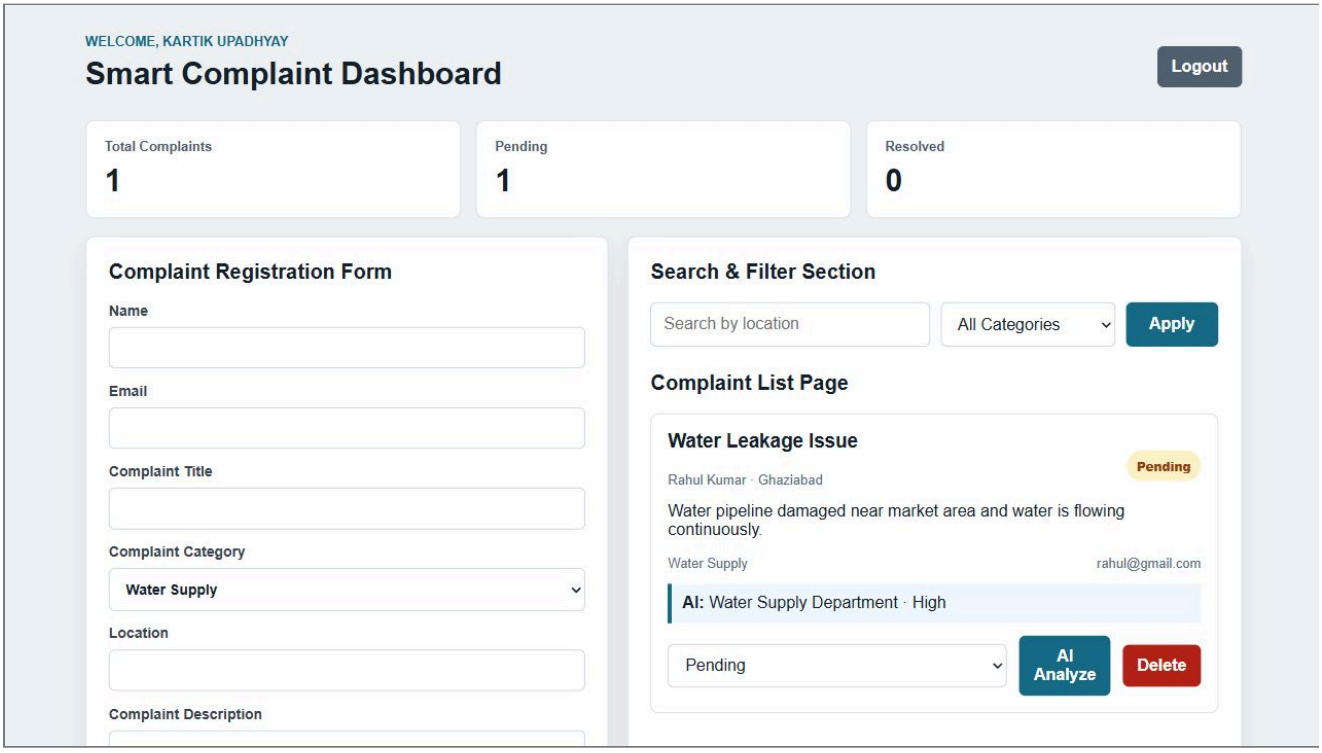
Login Page



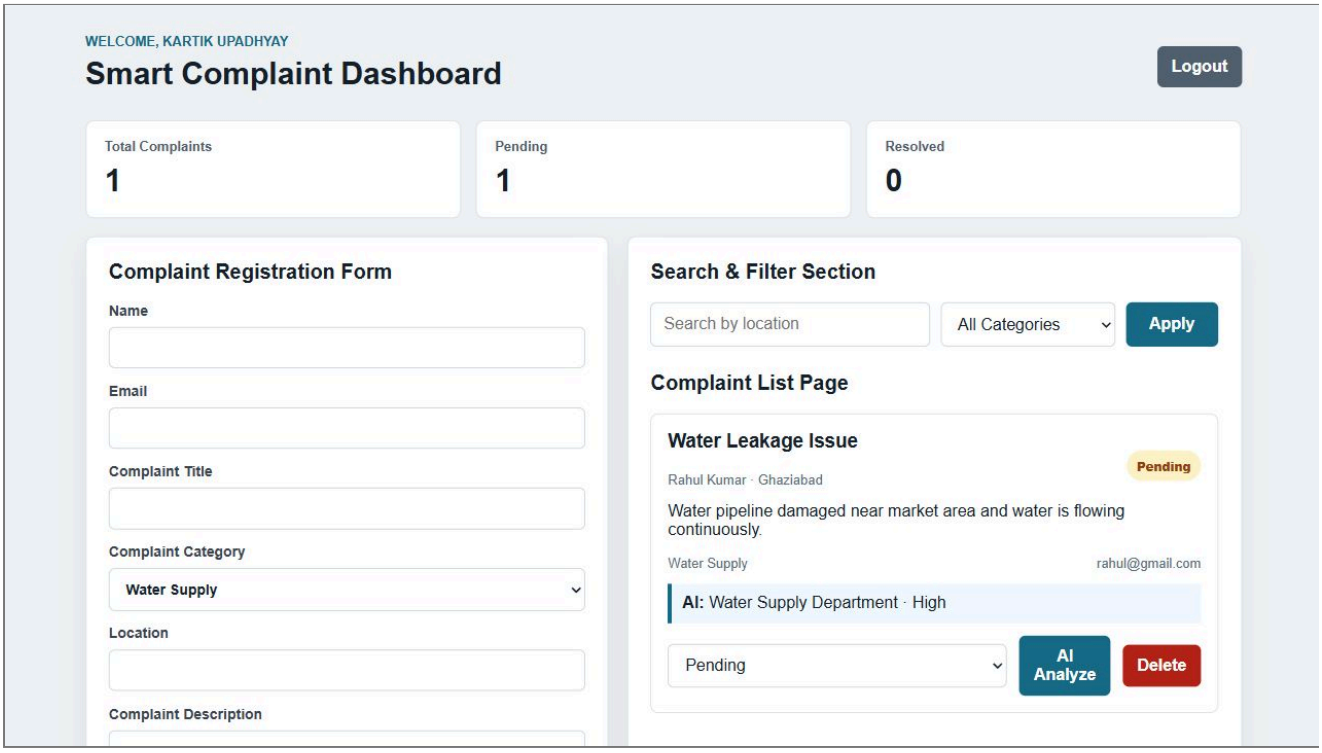
Dashboard / Complaint List Page



Complaint Registration Form



AI Analysis Result Display



Complaint Status Update Page

Paste Status Update Screenshot Here

MongoDB Atlas Users Collection

Paste MongoDB Atlas Users Collection Screenshot Here

MongoDB Atlas Complaints Collection

Paste MongoDB Atlas Complaints Collection Screenshot Here

Render Backend Deployment Screenshot

Paste Render Backend Deployment Screenshot Here

Render Frontend Deployment Screenshot

Paste Render Frontend Deployment Screenshot Here

Screenshot of VS Code Project Structure

```
C:\Users\KARTIKK\Desktop\Cracks\smart-complaint-management-system
|-- backend
|   |-- config
|   |   -- db.js
|   |-- controllers
|   |   |-- aiController.js
|   |   |-- authController.js
|   |   -- complaintController.js
|   |-- middleware
|   |   |-- authMiddleware.js
|   |   -- errorMiddleware.js
|   |-- models
|   |   |-- Complaint.js
|   |   -- User.js
|   |-- routes
|   |   |-- aiRoutes.js
|   |   |-- authRoutes.js
|   |   -- complaintRoutes.js
|   |-- .env.example
|   |-- package.json
|   -- server.js
|-- frontend
|   |-- src
|   |   |-- components
|   |   |   -- ProtectedRoute.jsx
|   |   |-- context
|   |   |   -- AuthContext.jsx
|   |   |-- pages
|   |   |   |-- Dashboard.jsx
|   |   |   |-- Login.jsx
|   |   |   -- Register.jsx
|   |   |-- api.js
|   |   |-- App.jsx
|   |   |-- index.css
|   |   -- main.jsx
|   |-- .env.example
|   |-- package.json
|   -- vite.config.js
|-- README.md
-- .gitignore
```

Paste VS Code Project Structure Screenshot Here

Manual Process Required

1. Paste secrets in backend/.env: MONGO_URI, JWT_SECRET, and OPENROUTER_API_KEY. Do not commit .env.

2. Push to GitHub: initialize git, commit, add remote, and push main branch.

3. Deploy backend on Render: Root Directory backend, Build Command npm install, Start Command npm start.

4. Deploy frontend on Render: Root Directory frontend, Build Command npm install && npm run build, Publish Directory dist.

5. Add frontend env on Render: VITE_API_URL=https://your-backend.onrender.com/api.

6. Add screenshots: MongoDB Atlas collections, Postman/Thunder requests, Render success pages, and VS Code project structure.